MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Macari Chiril-Adam, FAF-243
Carazanu Augustin, FAF-243

Report

Lab Work No.2

Network Programming

A Coffee Shop Server

Checked by:
asist. univ. Cristofor Fistic

Chișinău – 2026

1. Purpose of the work

The goal of this laboratory work is to build the server of a coffee shop directly on TCP sockets. Clients connect, ask for the menu and order products; the server takes the products from its stock, prepares the orders with a fixed number of workers and tells each client when its order is ready. A provider connects to bring new products and is alerted by the server when a product runs low. The server must serve many clients at the same time, protect the stock they share, and keep working for everybody else when one connection fails or sends invalid data.

The work was done in a team of two: **Macari Chiril-Adam** and **Carazanu Augustin** (FAF-243). The source code and the instructions to build and run it are in the repository https://github.com/chiriladam/network-programming-labs (folder `lab2-coffee-shop`).

2. Environment and requirements

Table 1 – Machine and software used for the runs shown in this report

Component	Value
CPU	AMD Ryzen 5 5600H, 6 physical cores / 12 logical processors
RAM	16 GB DDR4-3200
Operating system	Windows 11 Pro, version 10.0.26200, 64-bit
Language and runtime	Java, OpenJDK 25.0.1
Workers of the server	12 (one per logical processor, `Runtime.availableProcessors()`)

Only the socket API of Java is used: `ServerSocket` (the equivalent of socket, bind, listen and accept) and `Socket` (connect, send, receive, close) with their byte streams. No wrapper, HTTP, WebSocket, RPC or messaging library is used; the protocol and the cutting of the byte stream into messages are our own code. The server port, and the address and port the client and provider connect to, are command-line parameters. The client and the provider are one interactive console program that starts in either role.

3. Design

3.1 Architecture

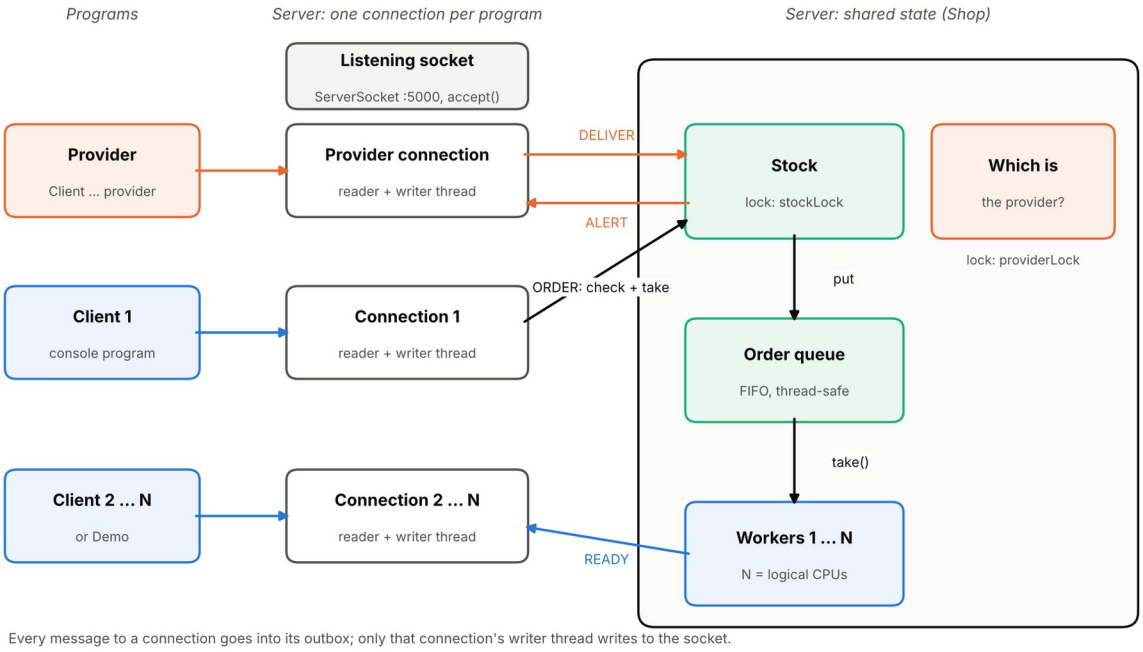


Figure 1 – Connections, threads and the shared state of the server

The main thread of the server creates the listening socket (`bind` on port 5000, backlog 100) and then calls `accept()` forever. For every accepted connection it starts a `ClientSession` with **two threads**: a *reader* that receives messages and executes the commands, and a *writer* that is the only thread that ever writes to that socket. Every other thread that has something to tell this connection (a worker with READY, another client's thread with an ALERT for the provider) only puts a line into the connection's *outbox* queue. The shared state is in one object, `Shop`; the orders are prepared by **12 worker threads** that take them from a FIFO queue.

Table 2 – Source files

File	Role
`Server.java`	bind, listen, accept forever; starts the workers and the stock saver
`ClientSession.java`	one connection: reader thread (commands) and writer thread (all output)
`LineReader.java`	cuts the received bytes into messages at '\n'; limits their length
`Shop.java`	stock, order queue, provider; check-and-take of the stock in one critical section
`Order.java`	one order and its state: WAITING → PREPARING → READY or CANCELLED
`Worker.java`	takes the oldest order, waits its preparation time, sends READY
`StockStore.java`	bonus: saves the stock to `stock.txt` and loads it at start
`Client.java`, `ShopConnection.java`	console client/provider: keyboard thread, receiver thread, heartbeat
`Demo.java`	many real clients at once: `many` (more clients than workers) and `rush` (last croissant)
`Product.java`, `Log.java`	the three products and their times; one log line per event

3.2 Protocol

The protocol is text over TCP: **one message per line, ended by '\n'** ('\r\n' is accepted too), at most 1024 bytes. Commands are not case-sensitive. Every answer starts with a keyword, so a program can recognise it by its first word.

Table 3 – Messages

Client → server	Server → client
`HELLO CLIENT` / `HELLO PROVIDER` (first message)	`OK you are a client` / `OK you are the provider`
`MENU`	`MENU coffee 50 in stock (4s), tea 15 in stock (3s), croissant 34 in stock (2s)`
`ORDER coffee 1 croissant 2`	`ACCEPTED order #1: 1 coffee, 2 croissant, preparation time 8s`, later `READY order #1 (...)`
	or `REFUSED not enough croissant: you asked for 3, only 1 left`
`DELIVER coffee 20` (provider only)	`DELIVERED 20 coffee. MENU ...`; from a client: `ERROR only the provider can deliver`
`HELP` / `QUIT`	the list of commands / `BYE` and the connection is closed
`PING` (every 5 s, hidden)	`PONG`
anything invalid	`ERROR <reason>`
(sent by the server on its own)	`READY order #N` to a client, `ALERT tea is low: 4 left` to the provider

3.3 Running the programs

`scripts\server.ps1 -Port 5000` builds and starts the server; `scripts\client.ps1` and `scripts\provider.ps1` start the console program as a client or as the provider (the same as `java -cp out Client localhost 5000 [provider]`); `scripts\demo.ps1 -Mode many -Clients 15` starts 15 clients at once. When it starts, the server prints its addresses, its 12 workers and where the stock came from (Figure 2).

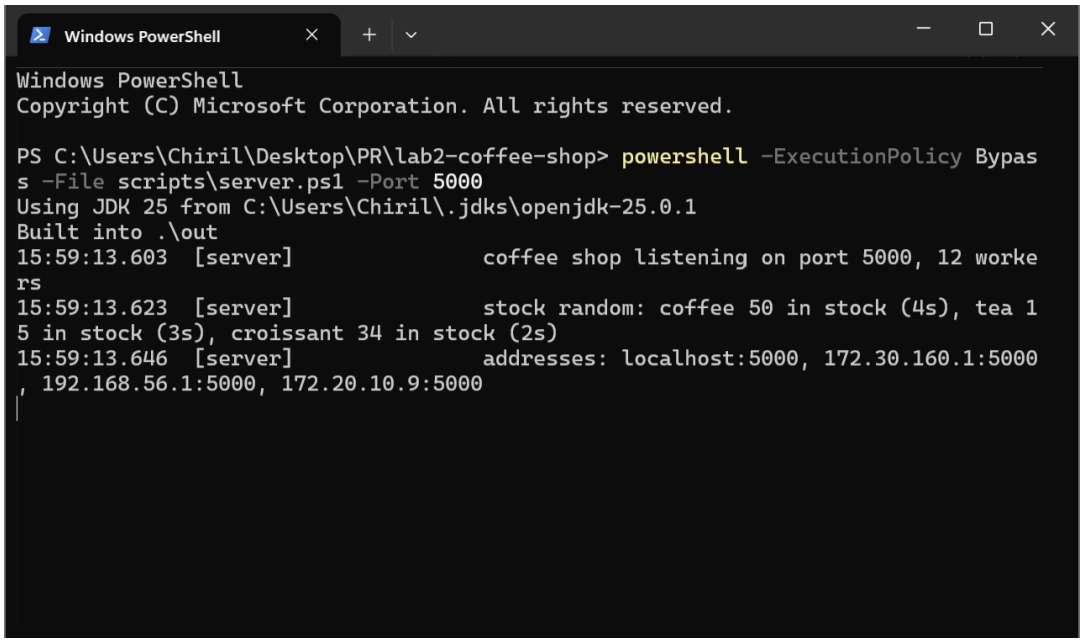


Figure 2 – The server started on port 5000 with 12 workers and a random stock

4. Task 1: one client

A client connects, asks for the menu and orders. If the order asks for more of a product than the stock has, it is refused and the client is told how many are left. Otherwise the products are taken at once, the client gets the order number and its preparation time immediately, and READY when the order is prepared. The preparation time is the sum over the items (1 coffee + 2 croissants = 4 + 2·2 = 8 s). An unknown product or a quantity that is not a positive number gets an error message.

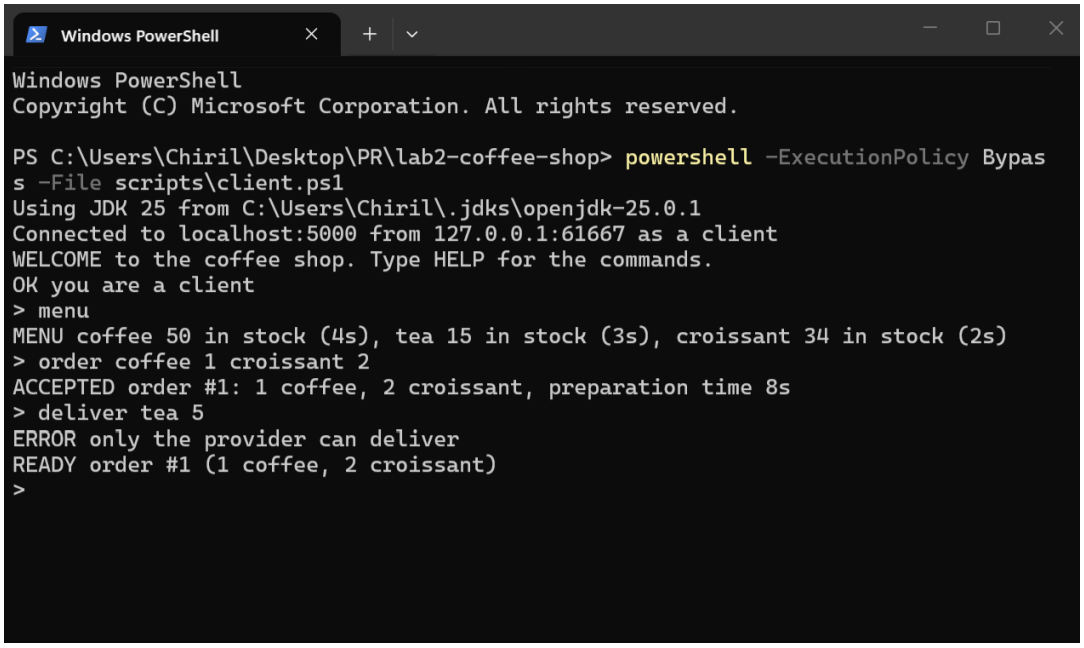


Figure 3 – A client: menu, an order of 8 s, a refused delivery, and READY 8 s later

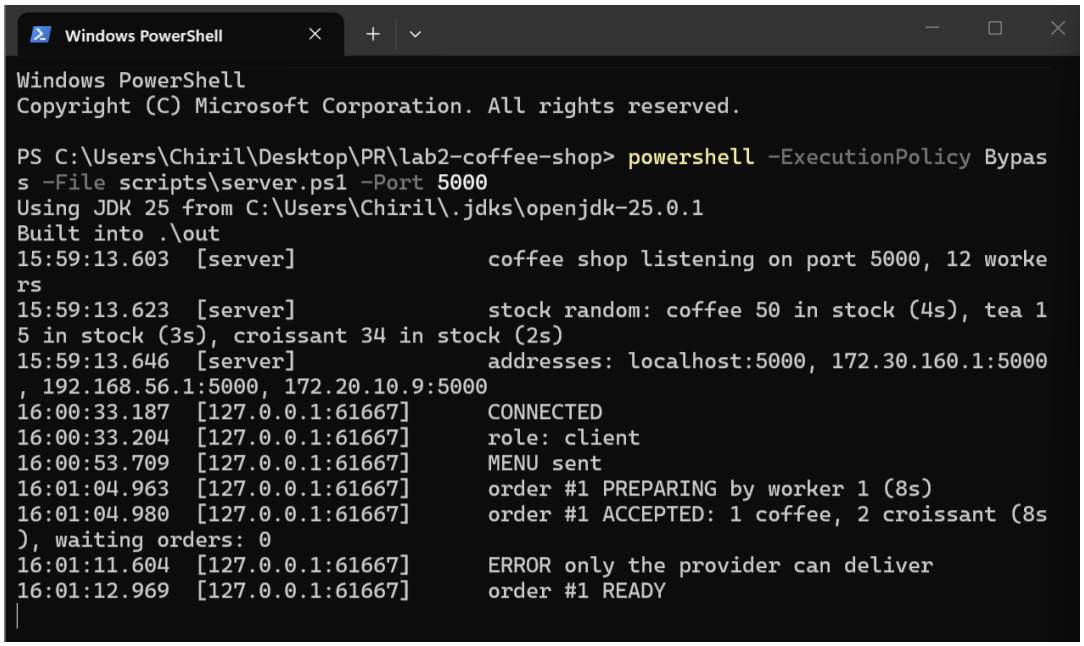


Figure 4 – The server log of the same session, with the endpoint of the client

Answers to the questions of Task 1

Q1. *How does your code cut the received bytes into messages? What does it do when a message arrives in two pieces, or two messages arrive in one receive?*

TCP delivers a stream of bytes, not messages, so the protocol ends every message with '\n' and `LineReader` cuts the stream there. It reads with `read()` into a 4 KB buffer and copies bytes into the current line until it meets '\n'; then it returns the line and keeps the rest of the buffer for the next call.

- **A message in two pieces** ("ME", then "NU\n"): after the first piece there is no '\n' yet, so the bytes stay in the current line and `readLine()` calls `read()` again; the message is returned only when its '\n' arrives.
- **Two messages in one receive** ("MENU\nMENU\n"): one `read()` fills the buffer with both; the first `readLine()` returns the first message, the second call returns the second one from the buffer without receiving again.
- **A message that is too long** (more than 1024 bytes without '\n') is not kept in memory: the client gets `ERROR message too long` at once and the rest, up to the next '\n', is skipped. We tested all three cases by sending bytes in pieces from a test script.

Q2. ***Why does the server answer at once, and tell the client later that the order is ready? What would the client see otherwise?***

Preparing an order takes seconds (here 8 s), and with more orders than workers it also waits in the queue. The answer *accepted or refused* is known at once, as soon as the stock is checked, so the client gets it immediately and can continue: ask for the menu, order again, or simply know that its order is in. If the server answered only when the order was ready, the client would see nothing for the whole preparation and waiting time and could not tell an accepted order from a lost connection; its reader thread on the server would also be blocked, so the client could not send anything else during that time. Separating the two answers is also what lets the workers, not the connection threads, decide how many orders are prepared at the same time.

5. Task 2: many clients

Every connection has its own threads, so clients connect at any time and each can order, ask for the menu or order again while its earlier orders are prepared. The server has one worker per logical processor, 12 on our machine; a free worker always takes the oldest waiting order (`LinkedBlockingQueue.take()`). To show it with more clients than workers, the demo starts **15 clients** that order 1 coffee (4 s) each at the same moment (Figure 5): 12 orders are prepared at once and are ready after about 4.2 s; the other 3 wait in the queue until a worker is free and are ready after 8.2 s. The server log shows the queue growing while the orders arrive (Figure 6).

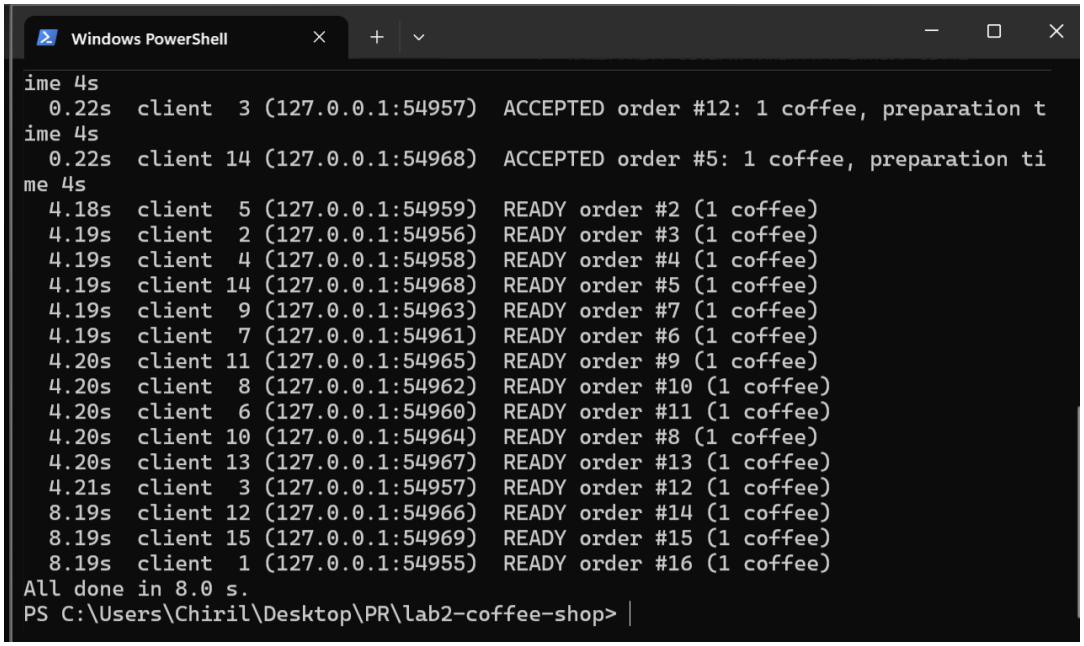


Figure 5 – 15 clients and 12 workers: 12 orders ready at 4.2 s, the last 3 at 8.2 s

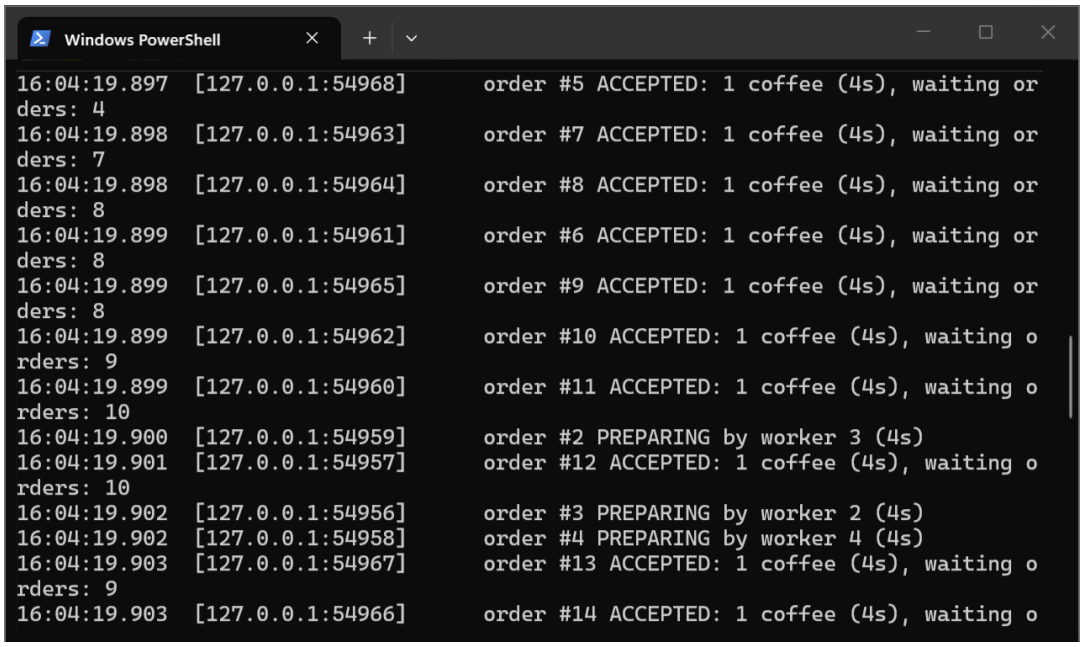


Figure 6 – The server log while the 15 orders arrive: the number of waiting orders grows

The protection of the stock is shown with the server started with **one croissant** in stock (`--stock coffee=20,tea=8,croissant=1`) and 10 clients that order a croissant at the same moment (Figure 7): exactly one order is accepted, the other nine are refused, and the stock goes from 1 to 0.

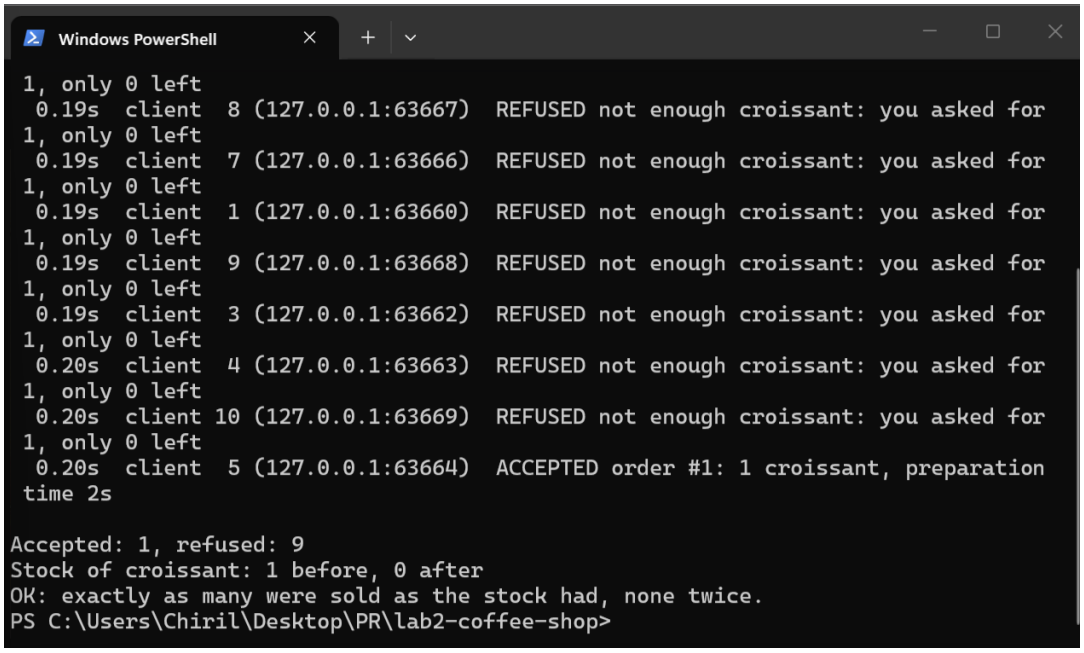


Figure 7 – 10 clients order the last croissant at the same moment: 1 accepted, 9 refused

Answers to the questions of Task 2

Q1. *Which data is shared between threads in your server, and how is each of them protected?*

Table 4 – Shared data and its protection

Data	Used by	Protection
stock of the 3 products	all client threads, the provider, cancellations	`synchronized (stockLock)`; check and take in one block
queue of waiting orders	client threads (put), 12 workers (take)	`LinkedBlockingQueue`: thread-safe and FIFO
next order number	client threads	changed only inside `stockLock`
state of one order	its worker and its client's thread	`synchronized` methods of `Order`
which connection is the provider	the provider's thread, all client threads (alerts)	`synchronized (providerLock)`
output of one connection	its reader, the workers, other clients' threads	one writer thread; others only add to its outbox
orders of one client	its reader thread, workers	`ConcurrentHashMap.newKeySet()`

Q2. *Two clients order the last croissant at the same moment. Show, step by step, what could go wrong without your protection.*

Without the lock, "is there enough?" and "take it" are separate steps, and two threads can interleave between them:

Table 5 – The last croissant without a lock (stock = 1)

Step	Thread of client A	Thread of client B	Stock
1	reads the stock: 1 ≥ 1, enough		1
2		reads the stock: 1 ≥ 1, enough	1
3	writes 1 − 1 = 0, answers ACCEPTED		0
4		writes 1 − 1 = 0 (from its old read), ACCEPTED	0

Both clients get the croissant, but only one existed. (If B re-read the stock before writing, it would write −1.) In our server the check of all items and the taking of all items happen inside **one** `synchronized (stockLock)` block in `Shop.placeOrder`: while A is inside, B waits at the start of the block; when B enters, it reads 0 and is refused. The rush demo shows it: of 10 simultaneous orders for 1 croissant, exactly one is accepted.

Q3. ***A worker tells a client that its order is ready while the thread of that client is reading from the same connection. What must you take care of?***

Reading and writing at the same time on one TCP socket is allowed: TCP is full-duplex and the input and output streams are independent, so the worker does not have to wait for the reader. What must be avoided is **two threads writing at the same time**: the bytes of two messages could be mixed ("REA" + "ACCEPTED" + "DY"), and a worker that writes directly to a slow or dead client could block, which would stop it from preparing other orders. So in our server no thread except the connection's own writer thread writes to the socket: the worker calls `send()`, which only adds the line to the connection's outbox and returns at once. We also make sure the client always receives ACCEPTED before READY: the ACCEPTED line is put into the outbox before the order enters the queue. If the client is already gone, `send()` simply drops the line.

Q4. ***Why one worker per CPU core? The workers spend their time waiting: what would happen with twice as many?***

The number of workers is the capacity of the shop, like the number of baristas: it limits how many orders are prepared at the same time, and the rule of the lab ties it to the machine (12 logical processors). If preparing an order were real CPU work, more workers than logical processors would not make it faster (Lab 1). But here a worker only *waits* (`wait(timeout)`), which uses no CPU. With **twice as many** workers (24), twice as many orders would be prepared at once: in the demo all 15 coffees would be ready after 4 s instead of 3 of them waiting until 8 s. The cost would be only memory for 12 more threads (their stacks), not CPU. So the limit is not a technical need here: it simulates a shop with a fixed number of baristas, and with real CPU-bound preparation it would also be the right number.

6. Task 3: the provider

The provider is the same console program started with the word `provider`; its first message is `HELLO PROVIDER`. Only that connection may send `DELIVER`, and a delivery is added to the stock at once (Figure 8, 49 → 69 coffees). A client that tries to deliver gets `ERROR only the provider can deliver` (Figure 3). Only one provider can be connected at a time; the provider cannot order.

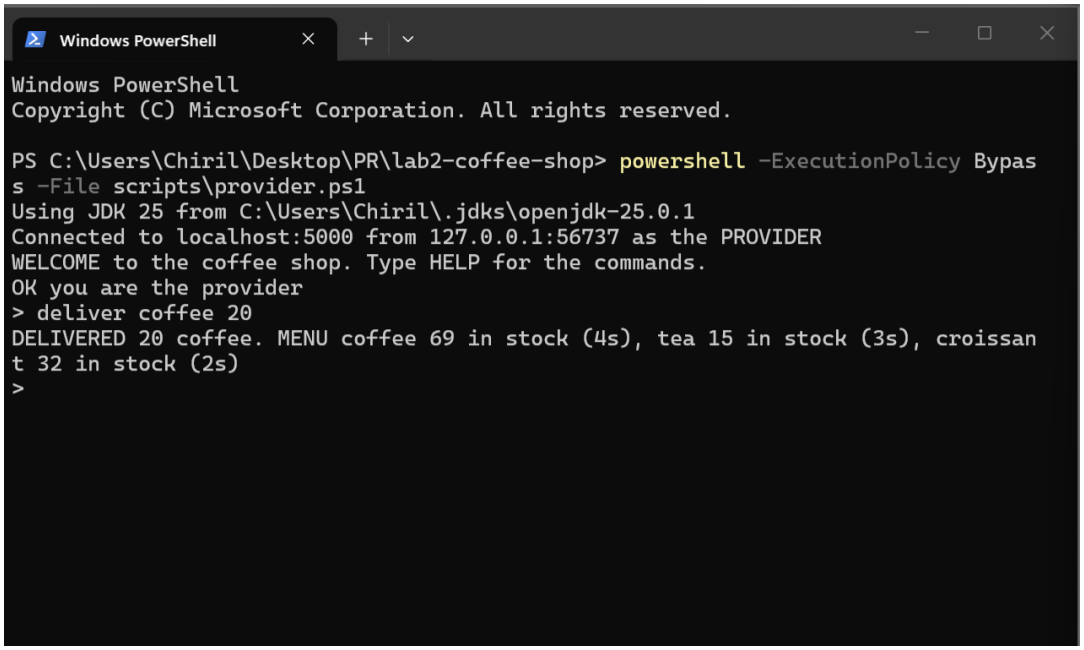


Figure 8 – The provider delivers 20 coffees; the answer shows the new stock

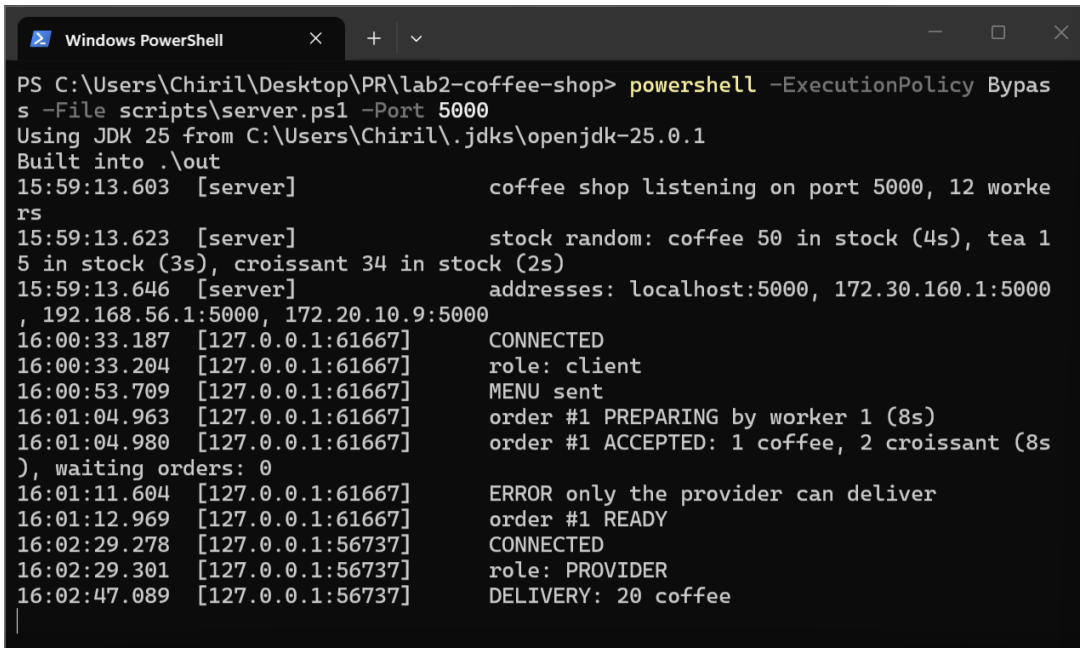


Figure 9 – The server log: the provider connects and the delivery is logged

When an order makes a product fall below 5, the server sends an ALERT to the provider at once, without a request. Figure 10 shows both cases. The server was started with 8 teas and 1 croissant, and the rush demo had sold the last croissant while no provider was connected. When the provider connected, it got `ALERT croissant is low: 0 left (while you were away)`. Then a client ordered 4 teas (8 → 4), and the provider got `ALERT tea is low: 4 left` at the same moment, without typing anything.

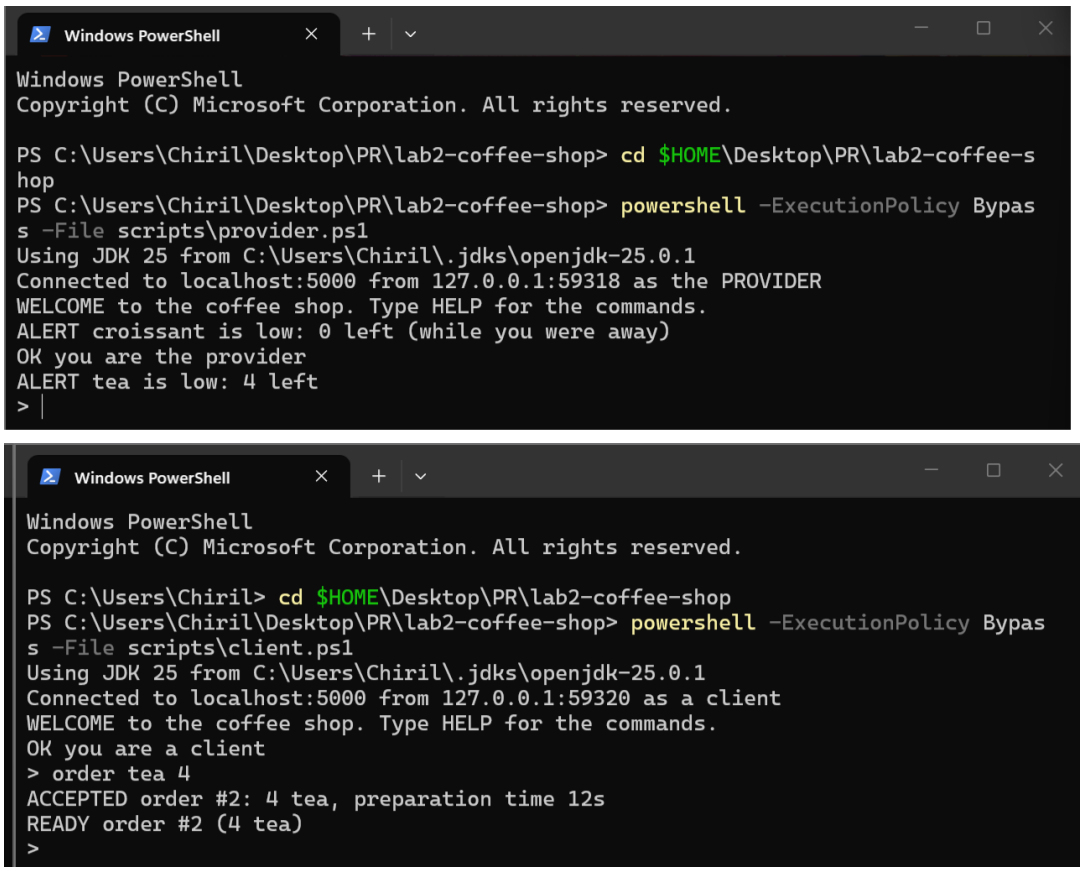


Figure 10 – Top: the provider gets the missed croissant alert on connect, then the tea alert. Bottom: the client's order that takes tea from 8 to 4

Answers to the questions of Task 3

Q1. *How does your server know which connection is the provider?*

By its first message. Our program sends `HELLO PROVIDER` (or `HELLO CLIENT`) right after connecting. The reader thread of that connection calls `Shop.registerProvider(this)`, which, under `providerLock`, stores this `ClientSession` as the provider if no other provider is connected; the session also remembers it in its `isProvider` flag. `DELIVER` is executed only when this flag is set; a second `HELLO PROVIDER` is refused, and the role cannot be changed later. When the provider disconnects, the reference is cleared and another provider can connect.

Q2. *A delivery and an order change the stock at the same moment. Show that no change is lost.*

Without protection, both would read the same old value and the second write would erase the first. With 50 coffees, an order of 3 and a delivery of 20 at the same moment:

Table 6 – Lost update without a lock (coffee = 50)

Step	Client's thread (order 3)	Provider's thread (deliver 20)	Stock
1	reads 50		50
2		reads 50	50
3	writes 50 − 3 = 47		47
4		writes 50 + 20 = 70	**70** (should be 67)

In our server both `placeOrder` and `deliver` change the stock only inside `synchronized (stockLock)`, so one of them waits until the other has finished: the result is 67 in either order. We checked it with a stress test: 40 clients ordering and disconnecting at random while the provider delivered, and the final stock was exactly initial + delivered − prepared, for every product.

Q3. ***Which thread sends the alert to the provider, and when?***

The alert is created by the **reader thread of the client whose order made the product fall below 5**. Inside the stock lock, `placeOrder` notes which products went from 5 or more to less than 5; right after the lock is released, it calls `alertProvider`, which puts the ALERT line into the provider's outbox. The **provider's writer thread** then sends it on the socket, at once. The alert is put into the outbox after the lock is released, so a slow provider can never hold up the stock. If no provider is connected, nothing is sent then; instead, when a provider connects, its own reader thread checks the stock and sends an alert for every product that is below 5.

7. Task 4: when things go wrong

When a connection ends, in any way, its reader thread leaves its loop and runs `disconnected()`: it cancels every order of that client, frees the provider role if it was the provider, and lets the writer thread close the socket. In Figure 11 the client window was closed while its order of 18 s was being prepared: the server noticed it at once ("Connection reset"), worker 6 stopped preparing the order, and the 2 coffees and 5 croissants went back to the stock. Orders that still waited in the queue are cancelled too; a worker that takes one later skips it. Whatever was prepared is lost: the next order that takes these products waits for their whole preparation again. When the provider disappears, the shop goes on with the stock it has.

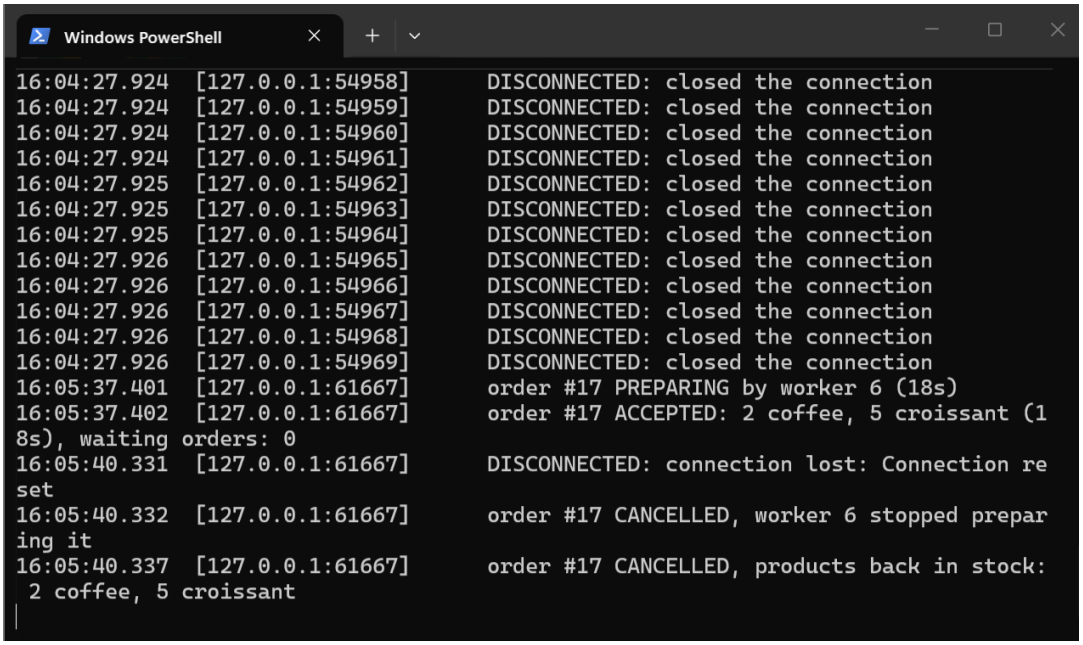


Figure 11 – A client disappears during an order of 18 s: the worker stops and the products go back

Table 7 – Invalid input and failures (all tested)

Situation	What the server does
unknown command, random bytes	`ERROR unknown command '...'` (non-printable bytes shown as \xNN in the log)
missing or wrong parameter	`ERROR every product needs a quantity`, `unknown product 'pizza'`, `must be a positive number`
message longer than 1024 bytes	`ERROR message too long`; the rest of the line is skipped, the connection stays usable
client closes or its process is killed	noticed at once (read returns −1 or "Connection reset"); its orders are cancelled
client's network gone	no data for 30 s (our client sends PING every 5 s): treated as gone, orders cancelled

client that never reads its messages	its outbox is limited to 1000 lines; then the connection is closed
any unexpected exception in one connection	caught in that connection's thread; the server and other clients go on
the server disappears	the client and the provider print the reason and end cleanly

Answers to the questions of Task 4

Q1. ***How does your server notice that a client has disappeared? Which kind of disappearance does it notice at once, and which one not?***

The reader thread of every connection is always blocked in `read()`, so it learns about the end of the connection from the result of that call:

- **The client closes the connection** (QUIT, Ctrl+C, closing the program normally): its operating system sends FIN, `read()` returns −1. Noticed **at once**.
- **The client's process is killed** or its window closed: the operating system closes the socket for it and sends FIN or RST; `read()` returns −1 or throws "Connection reset" (Figure 11). Noticed **at once**.
- **The client's network is gone** (Wi-Fi off, cable pulled, computer frozen): no packet arrives at all, so TCP has nothing to report and `read()` would wait for a very long time (TCP keep-alive is two hours by default). This one is **not noticed at once**. To notice it, our client sends `PING` every 5 s and the server sets `setSoTimeout(30 s)`: after 30 s of silence `read()` throws `SocketTimeoutException` and the client is treated as gone.

Q2. ***A client disappears while a worker is preparing its order. How does the worker find out and stop, and how do the products get back to the stock?***

The worker "prepares" by waiting inside `Order.prepare()` with `wait(timeout)` on the order object. The reader thread of the client, in `disconnected()`, calls `shop.cancel(order)` for each of its orders:

- `order.cancel()` (synchronized) changes the state from PREPARING (or WAITING) to CANCELLED and calls `notifyAll()`, which wakes the worker at once;
- the worker sees that the state is no longer PREPARING, `prepare()` returns false, the worker logs "stopped preparing it" and takes the next order from the queue;
- because `cancel()` returned true, `Shop.cancel` adds the items of the order back to the stock inside `stockLock`.

Finishing and cancelling at the same moment cannot both happen: both change the state inside the order's lock, so either the order becomes READY (the products were used) or CANCELLED (the products go back), never both.

Q3. ***Why must the client receive and read the keyboard at the same time? How does yours do it?***

Messages from the server arrive at any time, not only as answers: READY comes seconds after the order, an ALERT comes when another client orders, and the server can disappear while the user is thinking. Reading the keyboard blocks until the user presses Enter, so a program with one thread would show these messages only after the next line typed, and would not notice that the server is gone. Our client uses **three threads**: the main thread reads the keyboard and sends each line; a **receiver thread** (in `ShopConnection`) is always blocked in `read()` on the socket and prints every message as soon as it arrives, then shows the prompt again; a **heartbeat thread** sends `PING` every 5 s. When the

receiver sees that the connection ended, it prints the reason and ends the program with `System.exit(0)`, even while the main thread is waiting for the keyboard.

8. Bonus

8.1 The stock is kept when the server restarts

The stock is saved to `stock.txt` (one `product=quantity` line per product) after every change, and loaded when the server starts; only if there is no valid file is a random stock of 10–50 used. Client threads only call `requestSave()`, which sets a flag; one saver thread writes the file, always from a fresh snapshot of the stock, so the disk is never touched while the stock lock is held. The file is written to `stock.txt.tmp` and then renamed over `stock.txt`, so a crash in the middle of a write never leaves a broken file. We tested it by killing the server with `kill -9` after an order: on restart it printed "stock loaded from stock.txt" with exactly the stock after the order.

8.2 Two computers in the same network

The server listens on all network interfaces and prints the addresses other computers can use (Figure 2: for example `172.20.10.9:5000`). On the second computer the client connects with `scripts\client.ps1 -Server 172.20.10.9 -Port 5000`. On Windows the port must be allowed through the firewall, either with "Allow access" when the server first starts, or with `netsh advfirewall firewall add rule name="Coffee shop" dir=in action=allow protocol=TCP localport=5000`.

9. Testing

Besides the runs shown above, a test script connected to a running server with plain sockets and checked 35 cases, all passed: the menu; orders, refusals and preparation times; every error message; a message in two pieces, two messages in one send, a message of 5000 bytes and random bytes; a second provider; deliveries; an alert at once and alerts missed while the provider was away; cancellation on a normal close, on a connection reset and for orders still in the queue; ordering again while waiting; QUIT. A stress test with 40 clients that ordered and disconnected at random (half of them with a reset) while the provider delivered ended with the stock exactly equal to initial + delivered − prepared.

10. Conclusion

We built a coffee-shop server on plain TCP sockets in Java. A line-based protocol with our own framing handles messages that are split or merged by TCP. Each connection has a reader thread and a writer thread, the orders are prepared by one worker per logical processor from a FIFO queue, and the stock is changed only inside one lock, so two clients never get the same last croissant and a delivery never erases an order. The provider is alerted by the server itself, at once or when it connects. A client that disappears in any way has its orders cancelled and its products returned, and no invalid input or failure of one connection stops the server or the other clients. As a bonus, the stock survives a restart and the shop works between two computers.

11. Answers to the control questions

1. ***What is the difference between the listening socket and the sockets returned by accept? How many of each does your server have with five clients and the provider?***

The **listening socket** (`ServerSocket`) is bound to the server's port (5000) and only receives new connection requests; the operating system completes the TCP handshake and keeps the new connections in a queue (the backlog). It never carries data. Each call of `accept()` takes one connection from that queue and returns a **new socket**, connected to exactly one client (its address and port), which is used to send and receive that client's data. With five clients and the provider our server has **1 listening socket and 6 connected sockets** (and 2 threads for each connected socket).

2. ***Why is one send not one receive? How does your protocol deal with it?***

TCP is a byte stream: it guarantees the order of the bytes, not the boundaries of the sends. The sender's TCP may join several small sends into one segment (Nagle's algorithm) or split a large one into several segments; the network may deliver segments separately; and the receiver's `read()` returns whatever is in its buffer at that moment. So one send can arrive in several receives, and several sends in one receive. Our protocol marks the end of every message with '\n', and `LineReader` collects bytes until it meets '\n', returning exactly one message per call, however the bytes arrived (section 4, Q1).

3. ***What does it mean when receive returns 0? And when it fails because the connection was reset?***

When receive returns 0 (in Java `read()` returns −1), the other side has **closed the connection in an orderly way**: it sent FIN, all its data has been received, and nothing more will come. When receive fails with **"connection reset"**, the other side sent RST: the connection was aborted, for example because the process was killed or closed the socket with unread data, or the connection no longer exists there; data may have been lost. In both cases our server treats the client as gone and cancels its orders; only the log message differs ("closed the connection" or "connection lost: Connection reset").

4. ***Where in your server is a producer–consumer pattern? Name the producers, the consumers and the queue between them.***

- **Orders:** the producers are the reader threads of the clients (they put accepted orders), the queue is the `LinkedBlockingQueue<Order>` in `Shop`, and the consumers are the 12 workers (`take()` waits while it is empty).
- **Outgoing messages:** for every connection, the producers are its reader thread, the workers (READY) and the other clients' threads (ALERT for the provider); the queue is the connection's outbox, a `LinkedBlockingQueue<String>` limited to 1000 lines; the consumer is the connection's writer thread.
- A smaller one: client threads produce "save requests" (a flag) and the stock saver thread consumes them.

5. ***What does it cost when a thread waits for a client? What would change with 1000 clients?***

A thread blocked in `read()` uses no CPU, but it is not free: each thread has its own stack (Java reserves about 1 MB per thread on Windows, of which only a part is actually used), kernel data structures, and must be created and scheduled. Our server uses 2 threads per connection. With 1000 clients that is about 2000 threads: still possible, but it costs hundreds of megabytes of reserved memory, more context switches when many clients are active, and it approaches the limits of the

operating system; with 100 000 clients it would not work. Servers for many clients therefore use one thread for many sockets (non-blocking I/O with a selector: Java NIO, epoll, IOCP), or very cheap threads such as Java's virtual threads, which wait for the network without keeping an OS thread.

6. ***Explain step by step a race condition that could happen on your server without its locks.***

Two clients order the last croissant at the same moment (stock = 1). (1) A's thread reads the stock: 1, enough. (2) B's thread reads the stock: 1, enough. (3) A writes 0 and answers ACCEPTED. (4) B writes 0 and answers ACCEPTED. Two croissants were sold, one existed. Another one, between an order and a delivery, is shown in section 6, Q2: the delivery of 20 coffees overwrites the order of 3 and the stock becomes 70 instead of 67. With `synchronized (stockLock)` around the whole check-and-take, step 2 cannot start before step 3 has finished, so B reads 0 and is refused.

7. ***What identifies one TCP connection? How can five clients on the same machine all be connected to the same port of the server?***

A TCP connection is identified by four numbers: **(client IP, client port, server IP, server port)**. Each client socket gets its own local port, chosen by the operating system from the ephemeral range. Five clients on the same machine have the same client IP, server IP and server port 5000, but different client ports, so their four-tuples are different, and the operating system uses them to deliver each segment to the right socket. Figure 5 shows it: the 15 demo clients on one computer all connected to port 5000 from the ports 54955 to 54969.

12. Source code and how to run

Repository: https://github.com/chiriladam/network-programming-labs (folder `lab2-coffee-shop`); the README there describes every option. Requirements: JDK 17 or newer.

Table 8 – Commands (in the project folder)

What	Command
Build	`scripts\build.ps1` or `javac -d out src/*.java`
Server	`scripts\server.ps1 -Port 5000` or `java -cp out Server 5000`
Client	`scripts\client.ps1` or `java -cp out Client localhost 5000`
Provider	`scripts\provider.ps1` or `java -cp out Client localhost 5000 provider`
15 clients at once	`scripts\demo.ps1 -Mode many -Clients 15`
Last-croissant race	`scripts\demo.ps1 -Mode rush -Clients 10`
Server options	`--workers N`, `--stock coffee=10,tea=10,croissant=1`, `--stock-file f`, `--no-save`, `--log f`